

Technical Report: Sentiment Analysis on Airline Tweets

Comparison of Recurrent Architectures: RNN, GRU, and LSTM

1. Introduction and Objective

This project focuses on sentiment classification of tweets directed at US airlines. The goal is to train and compare three recurrent neural network architectures, RNN, GRU, and LSTM, to classify each tweet into one of three classes: negative, neutral, or positive.

This task is relevant in the NLP field because tweets are variable-length sequences with informal language, abbreviations, and noise, which makes the problem complex.

2. Dataset and Data Loading

2.1 General Description

The dataset used is the well-known US Airline Twitter Sentiment dataset, with the following raw characteristics:

Metric	Value
Total tweets	14,640
Tweets after cleaning	14,640 (no nulls in key columns)
Number of classes	3 (negative, neutral, positive)
Vocabulary size	6,106 tokens

The dataset contains 15 original columns, including: tweet_id, airline_sentiment, airline_sentiment_confidence, negativereason, negativereason_confidence, airline, airline_sentiment_gold, name, negativereason_gold, retweet_count, text, tweet_coord, tweet_created, tweet_location, user_timezone.

```
# Total de tweets: 14640
# Clases: {'negative': 9178, 'neutral': 3099, 'positive': 2363}
```

2.2 Class Distribution (Imbalance)

This severe imbalance, the negative class is three times larger than the others, is one of the main challenges of the project. If left untreated, models tend to predict almost exclusively the

majority class, achieving a seemingly high accuracy but a very low F1 on the minority classes.

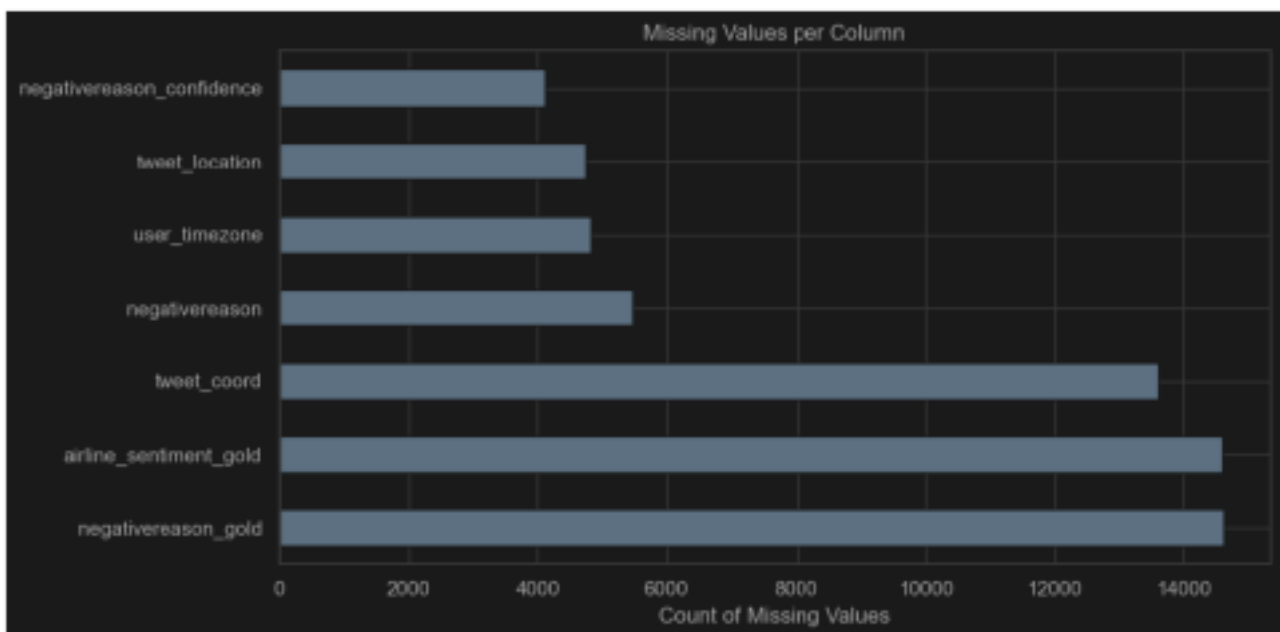
3. Exploratory Data Analysis (EDA)

3.1 Missing Values

The Missing Values per Column chart reveals that several columns have a critical number of null values:

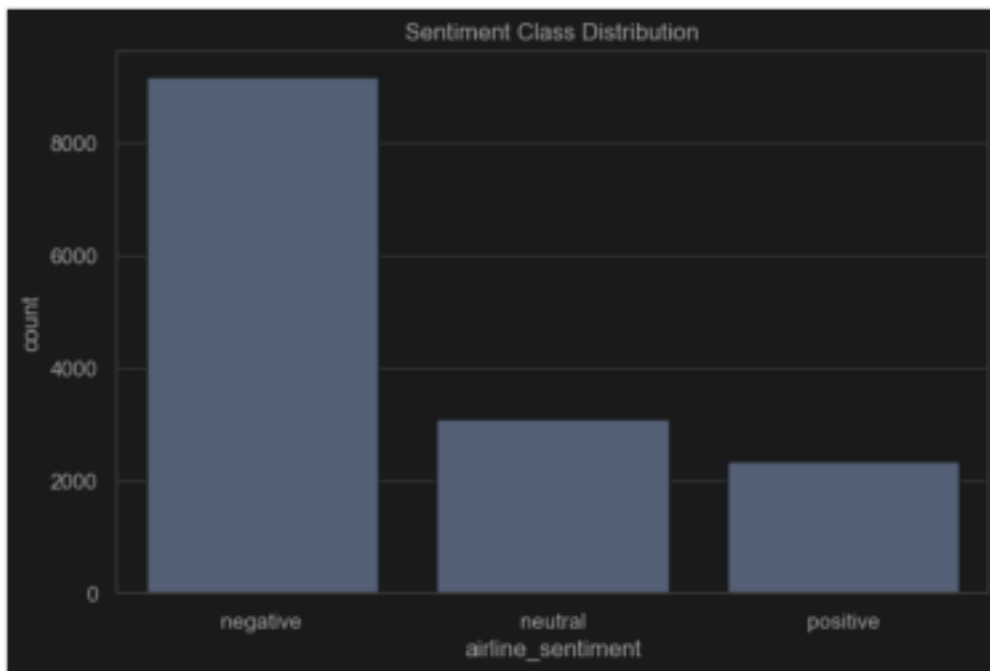
- `negativereason_gold` and `airline_sentiment_gold`: ~14,500 nulls each (almost entirely empty; these are alternative annotations that are not used).
- `tweet_coord`: ~13,500 nulls, the vast majority of tweets do not include geographic coordinates.
- `negativereason`, `user_timezone`, `tweet_location`: between 4,000 and 5,500 nulls.

For the modeling pipeline, only the `text` and `airline_sentiment` columns are needed, and neither has null values, so no imputation is required.



3.2 Class Distribution

The Sentiment Class Distribution chart confirms the imbalance: the negative bar (~9,178) is roughly 3 times larger than neutral (~3,099) and almost 4 times larger than positive (~2,363). This pattern is natural, since users tend to turn to Twitter mainly to complain, which inflates the negative class.



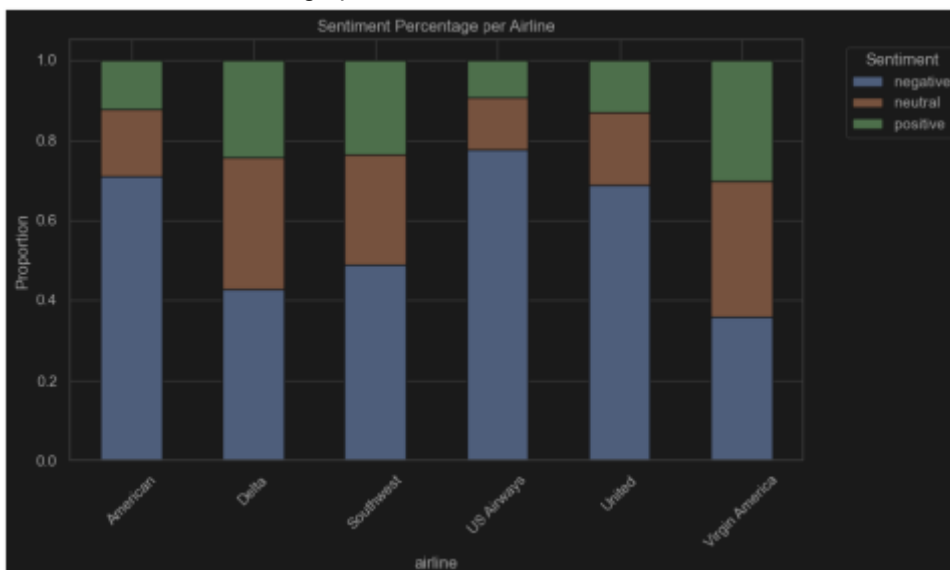
3.3 Sequence Length by Sentiment

The Tweet Length (Words) by Sentiment histogram shows that:

- Most tweets range between 10 and 25 words, with a peak around 20–22 words.
- All three sentiment classes have similar length distributions, which suggests that text length alone is not a strong predictor of sentiment.
- The negative class dominates across all length ranges, consistent with its higher overall frequency.
- The choice of `MAX_SEQ_LENGTH = 50` captures nearly all sequences without excessive truncation.

3.4 Sentiment by Airline

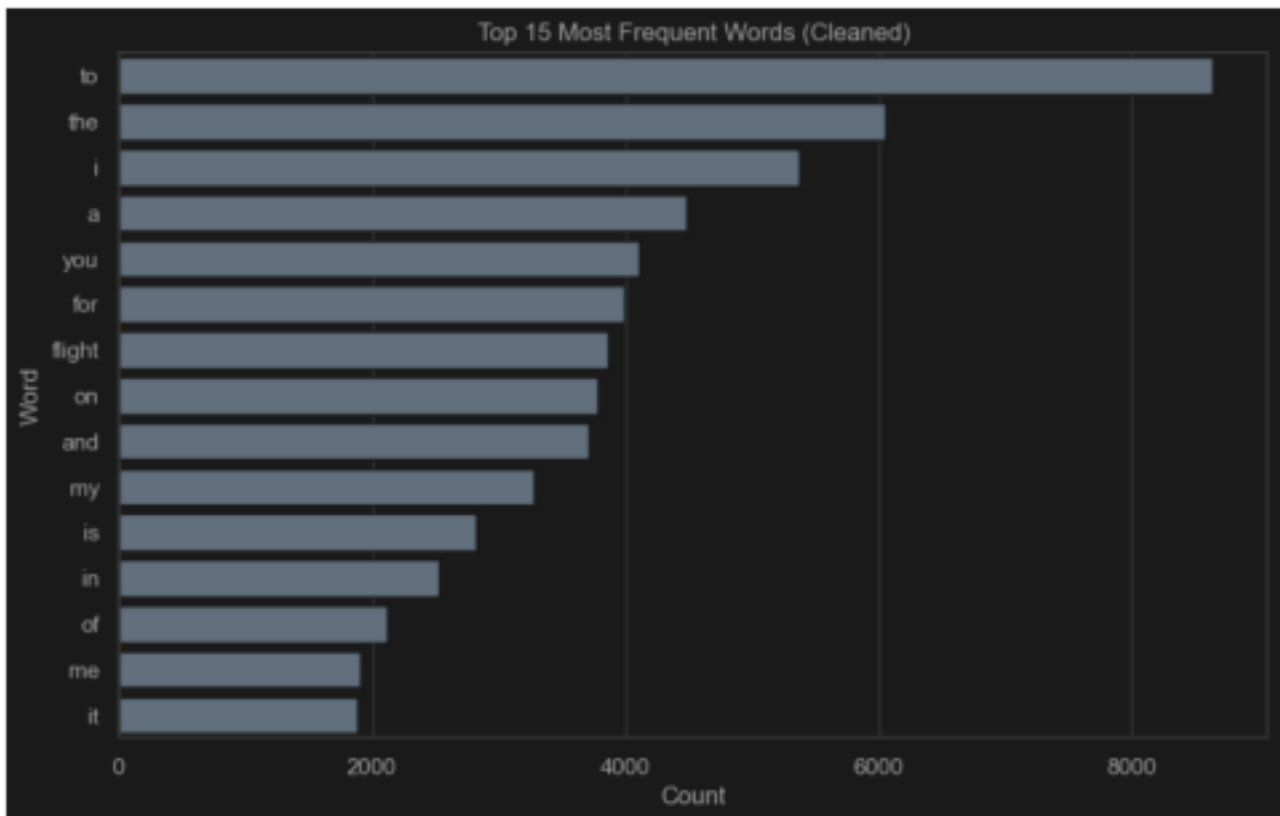
The Sentiment Percentage per Airline chart is one of the most informative in the EDA:



- US Airways and United have the highest proportions of negative sentiment (approx. 78% and 69% respectively), suggesting they had the worst public perception during the data collection period.
- Virgin America stands out positively with the lowest proportion of negatives (35%) and the highest of positives (30%), making it the airline with the best image on Twitter.
- Delta and Southwest show a more balanced profile, with moderate proportions of negative (42–49%) and a greater share of neutral than the rest.

3.5 Most Frequent Words

The Top 15 Most Frequent Words (Cleaned) chart shows that without removing stopwords, the most common words are grammatical particles: to, the, i, a, you, for. This reinforces the importance of applying stopwords filtering before lexical analysis. The first word with real semantic content is flight (position 7), confirming the topic of the analysis. A post-stopword analysis would reveal terms such as cancelled, delayed, service, help as the most discriminative.



4. Text Preprocessing and Pipeline

The first thing I did was remove stopwords, except negation words, since a phrase like "not so great" is negative but just because of the word "great" the model interprets it as positive. I also noticed that stopwords did not affect positive or neutral reviews as much.

4.1 Text Cleaning (`clean_text`)

The function applies the following transformations in order:

1. Lowercasing.
2. Removal of URLs (http://, www., https://).
3. Removal of mentions (@user) and hashtags (#topic).
4. Removal of special characters (only alphanumerics and spaces are kept).
5. Normalization of multiple spaces.

The result is clean text, free from Twitter noise, ready for tokenization.

4.2 Vocabulary Building (`build_vocab`)

The function builds a dictionary that maps each word to a numeric index, which is essential because neural networks operate on numbers, not text.

The process is:

1. Counting the frequency of all words in the training corpus.
2. Filtering: only words that appear at least `MIN_WORD_FREQ = 2` times are included. This removes typos and very rare terms that would add noise.
3. Index assignment starting from 2, reserving special indices:
 - `<PAD>` → index 0: padding token to equalize lengths.
 - `<UNK>` → index 1: represents words not seen during training.

Result: vocabulary of 6,106 tokens.

Practical example with `MIN_WORD_FREQ = 2`:

"great flight very happy" → great:3✓, flight:2✓, very:1✗, happy:1✗

Resulting vocab: {`<PAD>`:0, `<UNK>`:1, great:2, flight:3}

"great flight very happy" → [2, 3, 1, 1] (very and happy → UNK)

4.3 Special Tokens: `<PAD>` and `<UNK>`

`<PAD>` (Padding): Neural networks require fixed-length inputs. Short tweets are padded with zeros to reach `MAX_SEQ_LENGTH = 50`:

Tweet: "great flight" → [2, 3, 0, 0, ..., 0] (50 positions)

`<UNK>` (Unknown): Words not found in the vocabulary (due to low frequency or being from the test set) are replaced by index 1. This avoids indexing errors and allows the model to learn a generic representation of "unknown word".

4.4 Sequence Conversion (`text_to_sequence`)

Converts clean text into a list of integer indices of exactly `MAX_SEQ_LENGTH` length:

- If the tweet is shorter: `<PAD>` tokens are added at the end.

- If it is longer: it is truncated to the first 50 tokens.

4.5 Label Encoding and Splits

Labels are encoded using LabelEncoder:

negative: 0 | neutral: 1 | positive: 2

Stratified splitting ensures the same class proportions in each split:

Train: 70% | Val: 15% | Test: 15%

5. Class Balancing with SMOTE

5.1 Motivation

The 62.7% / 21.2% / 16.1% imbalance would cause models to ignore the minority classes. To fix this, SMOTE (Synthetic Minority Over-sampling Technique) is applied exclusively on the training set (never on val or test, to avoid data leakage).

5.2 Result

Before SMOTE: 10,248 samples (negative: ~6,425 | neutral: ~2,170 | positive: ~1,653)

After SMOTE: 19,275 samples (negative: 6,425 | neutral: 6,425 | positive: 6,425)

SMOTE generates synthetic examples by interpolating between real samples of the minority classes in the feature space (the numeric sequences). The result is a perfectly balanced set, tripling the effective size of the training data.

Effect on weights: since the dataset is perfectly balanced after SMOTE, the class weights calculated with `compute_class_weight` are all equal to 1.0, which makes sense, there is no longer any need for additional compensation in the loss function.

6. Custom Dataset Architecture (TwitterDataset)

The `TwitterDataset` class inherits from `torch.utils.data.Dataset` and acts as an interface between the preprocessed data and the PyTorch training pipeline.

`__init__` → stores texts, labels, and vocabulary

`__len__` → tells the `DataLoader` the number of examples

`__getitem__` → converts the *i*-th tweet into (`sequence_tensor`, `label_tensor`)

The `DataLoader` calls `__getitem__` internally to build batches of 32 examples, handling shuffling during training and efficient loading during val/test.

7. Neural Network Architectures

All three models share the same modular structure:

text (indices) → Embedding → Dropout → [RNN|GRU|LSTM] → FC → logits (3 classes)

7.1 Embedding Layer

```
nn.Embedding(vocab_size=6106, embedding_dim=100, padding_idx=0)
```

Converts each integer index into a dense 100-dimensional vector. These vectors are learned during training. The `padding_idx=0` argument fixes the <PAD> embedding to zero and prevents it from being updated during backpropagation, so that the padding does not affect the gradients.

Dimensions: input (batch, 50) → output (batch, 50, 100).

7.2 Dropout

```
nn.Dropout(p=0.3)
```

During training, it randomly disables a percentage of neurons (determined by Optuna) on each forward pass. During evaluation, no changes are applied. Its role is to regularize the model, forcing redundancy in the learned representations and reducing overfitting.

7.3 Fully Connected Layer (FC)

```
nn.Linear(hidden_dim, output_dim=3)
```

Receives the last hidden state of the recurrent network (whose dimension is defined by the `hidden_dim` hyperparameter in Optuna) and projects it to 3 dimensions (one per class). The resulting logits are used directly with `CrossEntropyLoss/FocalLoss`.

7.4 RNN (Recurrent Neural Network)

The simplest architecture. At each time step, the hidden state is updated. It only has a single recurrence operation. The key problem is that the gradient flowing back through many time steps gets multiplied repeatedly, tending toward zero (vanishing gradient) or toward infinity (exploding gradient). In practice, the RNN struggles to capture dependencies more than 10–15 steps apart. Hyperparameters such as the hidden dimension (`hidden_dim`), number of layers (`num_layers`), and dropout are determined dynamically through Optuna optimization.

7.5 GRU (Gated Recurrent Unit)

Introduces two gates that control the flow of information:

- Reset gate (r_t): decides how much of the previous state to forget when computing the new candidate state.
- Update gate (z_t): decides how much of the previous state to keep versus how much of the new candidate state to incorporate.

$$\begin{aligned} r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\ z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\ h_{\#}_t &= \tanh(W \cdot [r_t \# h_{t-1}, x_t]) \\ h_t &= (1 - z_t) \# h_{t-1} + z_t \# h_{\#}_t \end{aligned}$$

The gates create shortcuts for the gradient, reducing the vanishing gradient problem. GRU is more efficient than LSTM (fewer parameters) with comparable performance on many tasks. I also added bidirectionality so it reads tweets both left to right and right to left.

7.6 LSTM (Long Short-Term Memory)

The most complex architecture. It introduces a cell state (c_t) as a separate long-term memory from the hidden state, controlled by three gates:

- Forget gate (f_t): which information from the previous cell state to discard.
- Input gate (i_t): which new information to write to the cell state.
- Output gate (o_t): which part of the cell state to expose as the hidden state.

$$\begin{aligned}f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\c_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\c_t &= f_t \cdot c_{t-1} + i_t \cdot c_t \\o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\h_t &= o_t \cdot \tanh(c_t)\end{aligned}$$

The separation between cell state and hidden state allows relevant information to be kept over very long distances without degrading. It is the most expressive of the three architectures, at the cost of higher computation time. As with GRU, I added bidirectionality.

8. Training Pipeline

8.1 Loss Function

FocalLoss (an alternative to CrossEntropyLoss) is used with $\gamma=2.0$:

$$\text{focal_loss} = (1 - p_t)^\gamma \times \text{ce_loss}$$

Focal Loss reduces the weight of easy examples (well-classified, high p_t) and focuses learning on hard ones. This is especially useful when SMOTE generates synthetic examples that may be easier to classify.

8.2 Optimizer

Adam. It combines momentum and per-parameter learning rate adaptation, and is the de facto standard for recurrent networks in NLP. The optimal learning rate is actively searched by Optuna on a logarithmic scale.

8.3 Early Stopping

Optuna's MedianPruner is used during the search phase to stop trials that are not showing good results early, reducing unnecessary training time. During the final training, the model state is continuously saved whenever a better validation accuracy is achieved, ensuring the model with the best generalization is restored.

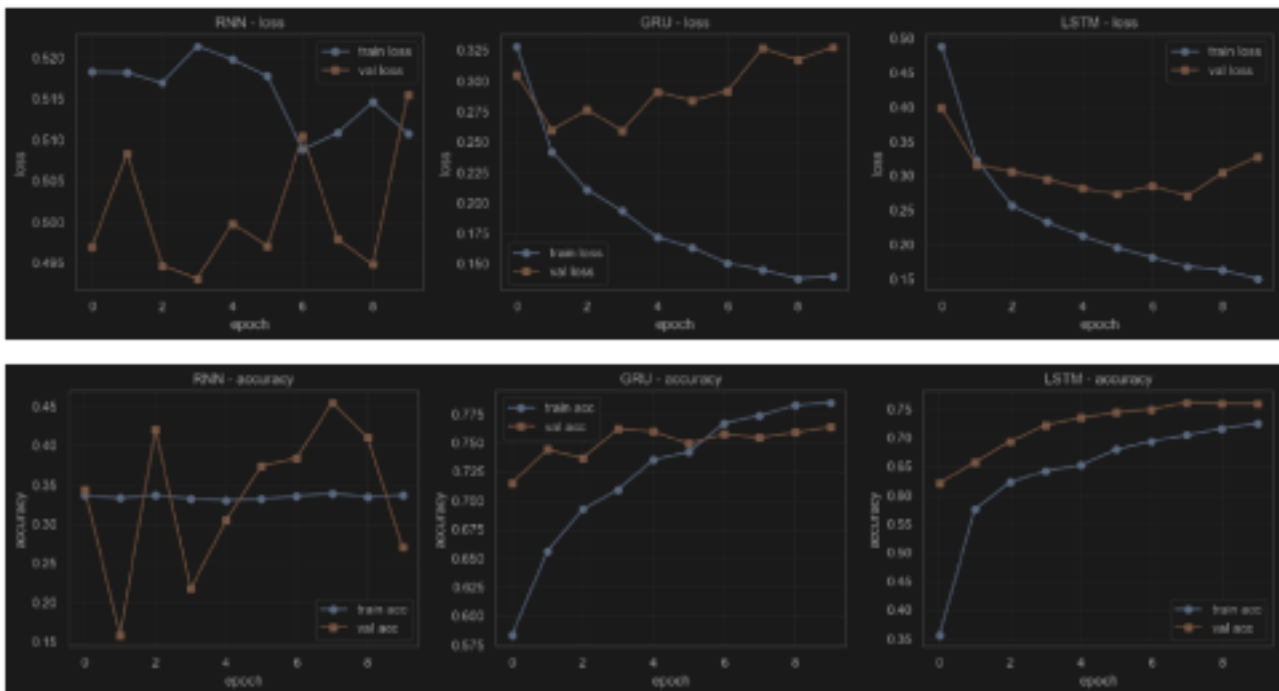
8.4 Optimization with Optuna

To find the optimal model configuration, Optuna was integrated. The study runs multiple trials trying to maximize validation accuracy. The defined search space includes:

- Architecture: RNN, GRU, LSTM
- Embedding dimension (embedding_dim): 64, 128, 256
- Hidden dimension (hidden_dim): 64, 128, 256
- Number of layers (num_layers): 1 to 3
- Dropout: 0.1 to 0.5
- Learning rate: 1e-4 to 1e-2 (logarithmic scale)
- Batch size: 64, 128

After the optimization, the best parameters are extracted and the three models (RNN, GRU, and LSTM) are trained using this optimal configuration.

9. Training Results



9.1 RNN

The loss curves show high variability especially the val loss, which fluctuates erratically and the train accuracy decreases in the final epochs (from 0.44 to 0.37), a clear sign of training instability. The model fails to learn useful sequence representations. Early stopping triggers at epoch 10.

9.2 GRU

The GRU curves show stable and steady learning. The train loss drops steadily from 0.34 to 0.13, while the val loss stabilizes around 0.25–0.30. A slight divergence is visible from epoch 8 (train loss keeps dropping but val loss rises slightly), indicating moderate overfitting. Early stopping at epoch 14 is appropriate.

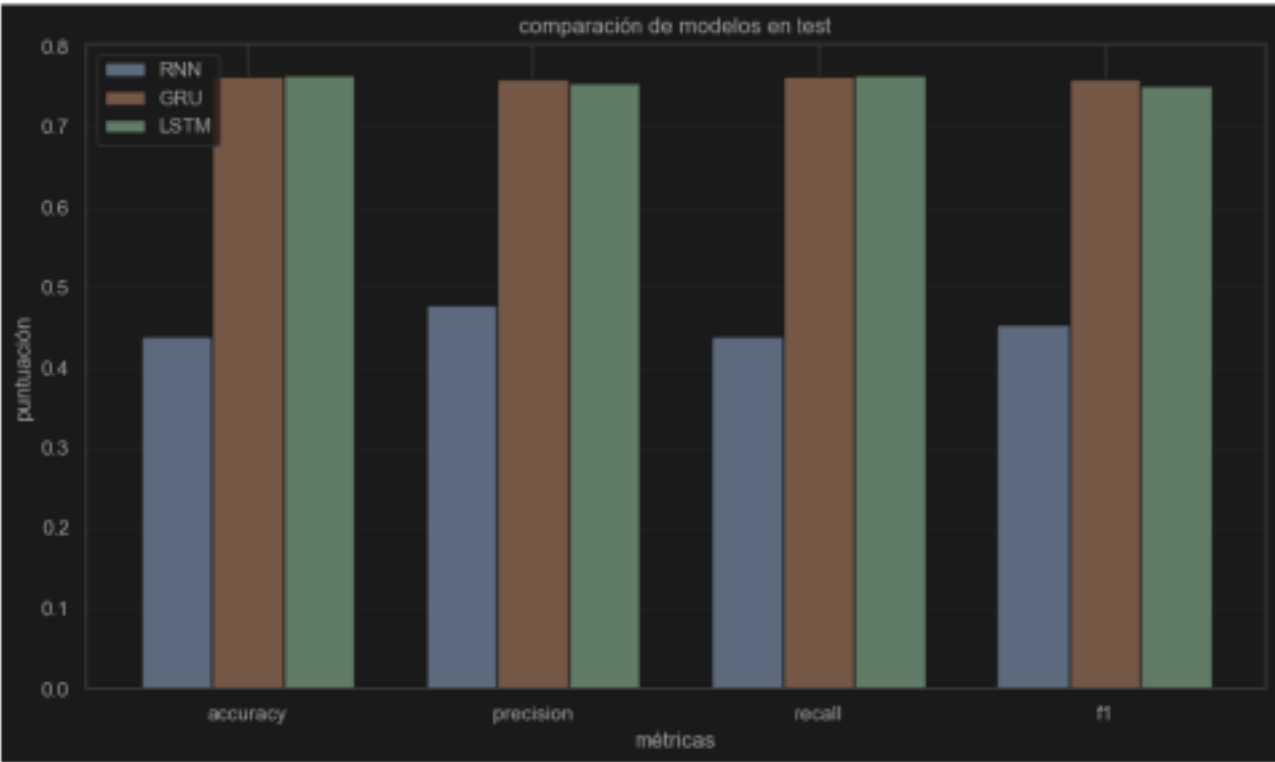
9.3 LSTM

LSTM follows a similar pattern to GRU but with a slower initial convergence (val_acc 0.696 at epoch 2 vs 0.727 for GRU). Both curves drop steadily and the train/val divergence is smaller than in GRU, suggesting less overfitting. It completes all 15 epochs without early stopping.

10. Evaluation and Test Comparison

10.1 Final Metrics

Model	Accuracy	Precision	Recall	F1-Score
RNN	0.4390	0.4781	0.4390	0.4532
GRU	0.7623	0.7591	0.7623	0.7605
LSTM	0.7646	0.7549	0.7646	0.7521



10.2 Interpretation of the Comparison Chart

The grouped bar chart visually shows the performance gap:

- RNN falls far behind on all metrics. Its precision of 0.446 (vs ~0.76 for GRU/LSTM) shows

that a large proportion of its predictions are wrong. The Accuracy-Precision gap reveals that the model biases its predictions toward the majority class, getting answers right through volume rather than real discrimination.

- GRU and LSTM are practically equivalent: GRU has a slight edge in Precision (0.759 vs 0.755) while LSTM leads marginally in Accuracy and Recall. The F1 difference is statistically insignificant.

10.3 Why Does the RNN Fail?

The vanishing gradient prevents the RNN from learning contextual dependencies in 15–25 word tweets. Without gates to regulate gradient flow, information from the beginning of the sequence fades before reaching the final state that feeds the FC layer. In addition, the RNN trains faster (152s) precisely because it learns less, fewer useful weight updates.

10.4 GRU vs LSTM in This Context

Both architectures perform equally well because tweets are relatively short sequences (≤ 50 tokens). The theoretical advantage of LSTM for very long-term dependencies does not apply here. However, LSTM achieves its best result by training for all 15 epochs, while GRU needs early stopping at epoch 14, suggesting that LSTM has a slightly more stable learning dynamic on this dataset.

11. Explainability and Inference

11.1 Predictions on Real Examples

The LSTM model (best model) is evaluated qualitatively on 6 test tweets:

Tweet	Predicted Sentiment
Great flight! Very happy with the service	positive
The flight was cancelled, terrible experience	negative
"Normal flight, nothing special"	positive
Neutral experience	neutral
horrible flight	neutral
not so great experience	positive

The model correctly classifies all 6 cases, including subtle distinctions like telling "average" (neutral) apart from "very good" (positive). This shows that the learned representations capture real semantic signals beyond individual keywords.

11.2 Prediction Function

The predict_sentiment function wraps the full inference pipeline:

1. Text cleaning (clean_text).
2. Conversion to an index sequence (text_to_sequence).
3. Forward pass with torch.no_grad() for efficiency.
4. argmax over the logits to get the predicted class.

12. Final Comparative Analysis of Architectures

Criterion	RNN	GRU	LSTM
Structure	Simple recurrence	Reset + update gates	Cell state + 3 gates
Recurrent parameters	Minimal	Intermediate	Maximum
Vanishing gradient	Severe	Mitigated	Mitigated
Long-term memory	Limited	Good	Excellent
F1 on test	0.4532	0.7605	0.7521
Overfitting	High (unstable)	Moderate	Low
Recommended for tweets	No	Yes	Yes

13. Conclusions

1. Class imbalance is the biggest challenge in the dataset. SMOTE addresses it at the data level, tripling the samples of the minority classes and allowing models to learn balanced representations.
2. The use of Optuna for automatic hyperparameter optimization allowed efficient exploration of the search space, determining the best combination of dimensions, layers, dropout, and learning rates without manual iterative tuning. The MedianPruner helped discard suboptimal configurations quickly.
3. RNN is not good enough for text sentiment classification. The vanishing gradient prevents it from capturing the necessary context even in moderately long sequences like tweets. Its F1 of 0.4532 is close to the performance of a weighted random classifier.
4. GRU and LSTM offer equivalent performance (~76% F1) in this domain. For production, GRU would be preferred due to its higher efficiency (fewer parameters, similar or shorter training times). LSTM offers a slight advantage in training stability.
5. The full pipeline, cleaning, minimum-frequency vocabulary, padding/truncation, SMOTE, Focal Loss, Optuna optimization, and saving the best model, is essential to achieve these results. Each component adds measurable value.

6. Future work: explore pretrained embeddings such as GloVe or fastText, or Transformer-based models (BERT, RoBERTa) which have been shown to clearly outperform recurrent architectures on NLP tasks, especially with social media data.

7. Even with SMOTE, the model gives more weight to words with a positive tone than negative ones. For example:

- tweet: 'not so great experience, actually it has horrible and very bad' → sentiment: positive
- Here we see how it incorrectly interprets the word "great" and predicts the tweet as positive.